Licenciatura en Sistemas de Información

Programación Avanzada - 2026

PROGRAMACIÓN AVANZADA · PRÁCTICA DE LABORATORIO · CLASE 2 (LUNES)

SQL con Firebase SQL Connect: modelo relacional y autorización

PostgreSQL detrás de una API declarativa: el cliente nunca escribe SQL y cada operación decide quién puede ejecutarla.

Dato	Detalle
Duración	60 minutos en clase + ~20 minutos de preparación previa (obligatoria)
Modalidad	Individual o en parejas, sobre el mismo repo y proyecto de la clase 1
Stack	Next.js 16 (export estático) · Firebase Auth · SQL Connect (ex Data Connect) · PostgreSQL 17
Costo	Cero. Plan **Spark**: prueba sin costo de Cloud SQL por 90 días, sin tarjeta
Material	Repo `clase-2-lunes` (provisto por la cátedra)
Ejes	Modelado relacional · Autorización a nivel de operación · Seguridad de APIs

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

1. Contexto

El catálogo de cursos que desplegamos en la clase 1 tiene los datos **escritos en el código**. Ahora el negocio quiere que los alumnos puedan **crear una cuenta, inscribirse a cursos y ver sus inscripciones**. Eso exige una base de datos, usuarios autenticados y, sobre todo, reglas claras de quién puede leer y escribir qué.

La app sigue siendo **estática** (no hay servidor propio). El navegador habla directamente con **SQL Connect**, un servicio de Firebase que pone una API delante de PostgreSQL. La pregunta central de la práctica es:

> Si el código del cliente es público y cualquiera puede llamar a la API directamente, ¿qué impide que alguien lea los datos de otros o se inscriba en nombre de otra persona?

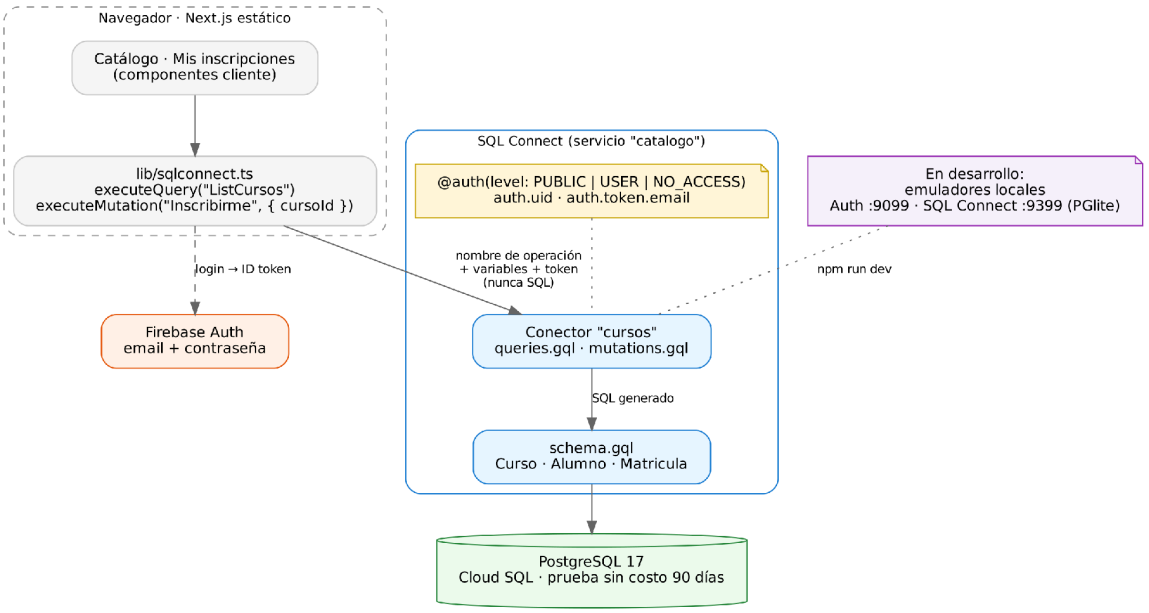
2. Objetivos de aprendizaje

1. Modelar una relación **muchos-a-muchos** con una tabla intermedia y clave primaria compuesta.
2. Entender por qué un cliente **no debe enviar SQL**: el servidor solo expone operaciones con nombre, definidas de antemano.
3. Declarar la **autorización** de cada operación (`PUBLIC`, `USER`, `NO_ACCESS`) y justificarla.
4. Usar **valores del servidor** (`auth.uid`, `auth.token.email`, `request.time`) en lugar de confiar en datos del cliente, y reconocer la vulnerabilidad **IDOR** cuando no se hace.
5. **Auditar** una API como lo haría un atacante, y corregir una fuga de datos personales.
6. Llevar el servicio a producción y ajustar la **CSP** de la clase 1 para que la app pueda hablar con él.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

3. Arquitectura



Componente	Responsabilidad	Dónde está
Firebase Auth	Identidad. Emite un token con `auth.uid` y `auth.token.email`.	Consola · `lib/firebase.ts`
Esquema	Tablas de PostgreSQL definidas en GraphQL con `@table`.	`dataconnect/schema/schema.gql`
Conector `cursos`	Las **únicas** operaciones que la app puede ejecutar, cada una con su `@auth`.	`dataconnect/cursos/*.gql`
`lib/sqlconnect.ts`	Invoca operaciones por nombre: `executeQuery(queryRef(dc, "ListCursos"))`. Ya escrito.	`lib/`
Emuladores	Auth y SQL Connect locales (PostgreSQL embebido con PGlite). `npm run dev` los usa.	`npm run emulators`
`scripts/atacar.mjs`	Auditoría: llama a tus operaciones como un atacante con la config pública.	`npm run atacar`

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

> **Decisión de diseño.** SQL Connect compila cada operación `.gql` a SQL en el servidor. El cliente manda solo **el nombre** de la operación y sus variables. No existe un endpoint que acepte SQL arbitrario, así que la inyección SQL queda cerrada por diseño, y la autorización se declara junto a cada operación, no en código disperso.
>
> **Prueba sin costo del plan Spark:** PostgreSQL 17 equivalente a una db-f1-micro, ~8.000 operaciones diarias, **90 días**, una sola vez por proyecto. Al vencer, la instancia se suspende y, si no se pasa a Blaze en el período de gracia, se borra.
>
> Durante la clase trabajamos con el **emulador local**: la prueba solo se activa al hacer el deploy de la Parte D.

4. Preparación previa (obligatoria, antes del lunes)

Si continuás tu repo de la clase 1

1. Commiteá todo lo que tengas.
2. Copiá sobre tu repo el contenido de `clase-2-lunes` **excepto** `.github/`, `.firebaserc` y `firebase.json` (ya están configurados con tu proyecto).
3. En **tu** `firebase.json`, agregá las claves `dataconnect` y `emulators` (el bloque está en el README).
4. Ejecutá `npm install`.

En la consola de Firebase

1. **Authentication** → *Comenzar* → *Sign-in method* → habilitar **Correo electrónico/contraseña**.
2. ⚙ *Configuración del proyecto* → *Tus apps* → crear una **app web** si no existe y copiar `apiKey`, `authDomain`, `projectId` y `appId` a `lib/firebase-config.ts`.

En tu máquina

1. `npm run emulators`: la primera vez descarga el emulador de SQL Connect. Tiene que terminar en *All emulators ready* con Auth en 9099 y SQL Connect en 9399.
2. En otra terminal: `npm run seed` (carga 7 cursos y 2 alumnos ficticios) y después `npm run dev`.
3. Abrir `http://localhost:3000/cursos`: tienen que aparecer los 7 cursos leídos desde PostgreSQL.
4. Recomendado: instalar la extensión **Firebase** de VS Code (valida y autocompleta los `.gql`).

> **Llegás a clase con:** los emuladores funcionando, el catálogo visible en `localhost:3000/cursos` y Email/contraseña habilitado en tu proyecto.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

5. Cronograma de la clase

Minuto	Bloque	Actividad
00 – 05	Introducción	Arquitectura: ¿por qué el cliente no manda SQL? Recorrido por `dataconnect/`.
05 – 10	Chequeo	Emuladores arriba, seed cargado, crear una cuenta en la app. Primer `npm run atacar`.
10 – 20	Parte A	L1: modelar `Matricula`.
20 – 35	Parte B	L2 `Inscribirme` y L3 `MisInscripciones`; probar en la UI. L5 si hay tiempo.
35 – 45	Parte C	L4: auditoría con `npm run atacar`, corregir la fuga, repetir hasta que todo pase.
45 – 55	Parte D	L6 CSP + L7 deploy de SQL Connect; frontend por Pull Request.
55 – 60	Cierre	Puesta en común: IDOR, mínimo privilegio y qué hace cada nivel de `@auth`.

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

6. Parte A · Modelado relacional

Abrí `dataconnect/schema/schema.gql`. `Curso` y `Alumno` ya están. Observá dos detalles antes de empezar:

- `Curso` no declara `id`: SQL Connect agrega una clave primaria **UUID** automáticamente.
- `Alumno.id` tiene `@default(expr: "auth.uid")`: al insertar, el **servidor** completa el id con el usuario autenticado.

L1 · Tabla `Matricula`

Modelá la relación muchos-a-muchos entre `Alumno` y `Curso` según los requisitos del comentario `TODO L1`: clave primaria compuesta por alumno y curso, y un campo `fecha` que complete el servidor. Un campo `alumno: Alumno!` genera la clave foránea `alumnoId` automáticamente.

Logro: la terminal de los emuladores no muestra errores al guardar, y en la UI de emuladores (`127.0.0.1:4000` → SQL Connect) aparece la tabla.

> **Para pensar:** ¿qué garantiza la clave compuesta que no garantizaría una clave UUID propia de `Matricula`? ¿Dónde preferís que viva esa regla: en el código de la app o en la base de datos?

7. Parte B · Operaciones con autorización

Antes de escribir, leé el ejemplo resuelto `UpsertAlumno` en `mutations.gql`. El cliente manda **solo** `nombre`; el id y el email salen del **token**, con `id_expr: "auth.uid"` y `email_expr: "auth.token.email"`.

L2 · `Inscribirme($cursoId: UUID!)`

- Solo para usuarios autenticados.
- Inscribe **al usuario que llama**. La operación no puede aceptar un id de alumno como variable.
- Debe ser **idempotente**: inscribirse dos veces no falla ni duplica.

L3 · `MisInscripciones`

- Solo para usuarios autenticados; devuelve únicamente las inscripciones del usuario que llama.
- Respetá la forma de respuesta que espera la app (está en el comentario `TODO L3`): `alumno { nombre matriculas { fecha curso { id titulo nivel horas } } }`.

L5 · `DarmeDeBaja($cursoId: UUID!)` (opcional)

Borra solo la inscripción del usuario que llama en ese curso.

Logro: en la app podés crear una cuenta, inscribirte a un curso, verlo en *Mis inscripciones* y darte de baja.

> **Pregunta clave:** ¿qué pasaría si `MisInscripciones` recibiera `$alumnoId` como variable y filtrara por ella? Ese error tiene nombre: **IDOR** (Insecure Direct Object Reference), y es de los más frecuentes en APIs reales.

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

8. Parte C · Auditoría de seguridad

Con los emuladores corriendo, ejecutá `npm run atacar`. El script toma la configuración pública de tu app, igual que lo haría un atacante leyendo tu bundle, y prueba:

Control	Qué intenta
Catálogo público	Leer `ListCursos` sin login (debe funcionar: es público por diseño).
Datos personales	Leer `ListAlumnos` sin login.
Inscripción anónima	Ejecutar `Inscribirme` sin token.
Idempotencia	Inscribir dos veces al mismo usuario en el mismo curso.
Aislamiento de lectura	El usuario B pide `MisInscripciones` después de que A se inscribió.
Aislamiento de escritura	El usuario B ejecuta `DarmeDeBaja` sobre el curso de A.

L4 · La operación heredada

En `queries.gql` hay una operación `ListAlumnos` que dejó un desarrollador anterior "para el panel de admin". El script va a marcarla como vulnerable. Analizala: ¿quién puede ejecutarla?, ¿qué devuelve?, ¿qué tipo de dato es? Corregila con el nivel de acceso adecuado y justificá tu decisión.

Logro: `npm run atacar` termina con *Todos los controles pasaron* (o solo con L5 pendiente si no la hiciste).

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

9. Parte D · Producción

L6 · CSP

En la clase 1 dejamos `connect-src 'self'`. Ahora la app de producción necesita hablar con Firebase Auth y con SQL Connect. Si no ajustás la política, el catálogo no carga y la consola muestra *Refused to connect*.

- Identificá qué dominios llama la app. Pista: hacé `npm run build && npm run serve`, abrí `/cursos` y mirá la pestaña *Network* y la consola.
- Agregá a `connect-src` **solo** los orígenes necesarios. No uses comodines amplios como `https://*.googleapis.com`: justificá por qué.

L7 · Deploy

```
npx firebase deploy --only dataconnect          # 1.ª vez: crea la instancia Cloud SQL de prueba
npx firebase dataconnect:execute dataconnect/seed_data.gql   # datos iniciales en producción
```

- El CLI va a mostrar los cambios de esquema (migración) y te va a pedir confirmación. Leé qué SQL va a ejecutar.
- La instancia tarda varios minutos en crearse; mientras tanto SQL Connect guarda los datos en una base temporal.
- El frontend se publica **como en la clase 1**: rama, Pull Request, preview, merge.

Logro: en el preview y en producción podés crear una cuenta e inscribirte.

> **Si no llegás en clase:** la Parte D queda como tarea. Lo importante del día son las Partes A, B y C.

10. Preguntas de reflexión

Respondelas en `REFLEXION.md` (breves y concretas):

1. ¿Por qué ninguna operación de este conector recibe el id del alumno como variable? ¿Qué es IDOR y cómo lo evita `auth.uid`?
2. `ListCursos` es `PUBLIC` y `ListAlumnos` terminó en `NO_ACCESS`. ¿Qué criterio usaste? ¿Cómo haría el "panel de admin" para listar alumnos?
3. ¿Qué ventaja tiene una clave primaria compuesta en `Matricula` frente a validar duplicados en el código del cliente?
4. La configuración de Firebase (`apiKey`, etc.) es pública. ¿Por qué eso no compromete la base de datos? ¿Qué capa faltaría para frenar a alguien que usa esa config desde un script? (pista: App Check).
5. Comparado con una API REST propia (Express + `pg`), ¿qué te da SQL Connect y qué perdés?
6. ¿Por qué no alcanzó con la CSP de la clase 1 y por qué conviene listar dominios exactos en `connect-src`?

Licenciatura en Sistemas de Información
Programación Avanzada - 2026

11. Entregables

#	Entregable	Formato
1	Repo con `dataconnect/` completo (L1–L4, y L5 si la hiciste)	URL
2	Salida de `npm run atacar` **antes** y **después** de corregir	Texto o captura en `REFLEXION.md`
3	PR mergeado con el cambio de CSP (L6)	URL
4	Producción funcionando con inscripción (L7) — puede entregarse como tarea	URL `web.app`
5	Respuestas a la reflexión	`REFLEXION.md`

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

12. Rúbrica de evaluación

Criterio	Pts	Insuficiente	Esperado	Destacado
Modelado (L1)	20	No compila o permite duplicados.	`Matricula` con clave compuesta y fecha del servidor.	Justifica la clave compuesta y propone índices o restricciones adicionales (p. ej., cupo).
Operaciones (L2, L3)	25	Fallan o reciben el id del alumno como variable.	Funcionan desde la UI; usan `auth.uid`; `Inscribirme` es idempotente.	Además L5, y explica `insert` vs `upsert`.
Autorización y auditoría (L4)	25	`ListAlumnos` sigue expuesta o la auditoría falla.	`npm run atacar` sin vulnerabilidades; corrección justificada.	Propone cómo resolver el caso del panel de admin sin exponer datos.
Producción (L6, L7)	20	CSP rota o con comodines amplios; sin deploy.	CSP con orígenes exactos; servicio desplegado y app funcionando.	Usa `dataconnect:sql:diff` y analiza la migración.
Reflexión	10	Incompleta o genérica.	Todas respondidas con fundamentos.	Compara con una API propia o con Firestore Rules.

Total: 100 puntos. Se aprueba con 60, con *Operaciones* y *Autorización* al menos en nivel *Esperado*.

Anexo A · Sintaxis de referencia

Necesito…	Sintaxis
Tabla con clave compuesta	`type X @table(key: ["a", "b"]) { a: A!  b: B! }`
Valor por defecto del servidor	`campo: Timestamp! @default(expr: "request.time")`
Nivel de acceso	`@auth(level: USER)` · `PUBLIC` exige `insecureReason: "..."`
Usar el UID en una escritura	`x_upsert(data: { alumnoId_expr: "auth.uid", ... })`
Buscar por clave con el UID	`alumno(key: { id_expr: "auth.uid" }) { ... }`
Filas relacionadas (1-a-N)	`matriculas_on_alumno { ... }` (con alias: `matriculas: matriculas_on_alumno`)
Borrar por clave	`x_delete(key: { alumnoId_expr: "auth.uid", cursoId: $cursoId })`

Licenciatura en Sistemas de Información

Programación Avanzada - 2026

Anexo B · Problemas frecuentes

Síntoma	Causa probable
`atacar`: *No hay conexión con los emuladores*	Falta `npm run emulators` en otra terminal.
El catálogo no carga en `npm run dev`	Emuladores apagados o falta `npm run seed`.
operation not found al inscribirse	La operación no existe todavía, o el `.gql` no compila: mirá la terminal de los emuladores.
unauthenticated	No hay sesión, o la operación es `USER` y se llamó sin login (¡correcto!).
Error de datos viejos tras cambiar el esquema	Borrar `dataconnect/.dataconnect/` y repetir `npm run seed`.
Producción: *Refused to connect*	Falta L6 (`connect-src`).
Deploy: aviso de prueba ya usada	La prueba es una por proyecto: avisá a la cátedra.

Anexo C · Referencias

- SQL Connect — documentación: firebase.google.com/docs/sql-connect
- SQL Connect — precios y prueba sin costo: firebase.google.com/docs/sql-connect/pricing
- Firebase Blog — No-cost trials for Data Connect (marzo 2026)
- OWASP API Security Top 10 — API1: Broken Object Level Authorization (IDOR)